



NCA Sponsored Training

Day 2 – Afternoon Session (Theory)

Asymmetric Cryptography

Ahmed Didouh

Madinah

13/01/2026



rc3.kaust.edu.sa





Outline



1. Asymmetric Encryption Overview

- *Introduction to asymmetric cryptography*
- *Advantages and Disadvantages*

2. RSA Algorithm

- *Basic properties*
- *Crypto system*
- *Example*

3. Real-world usage of Public Key Encryption

- *Public Key Infrastructures*
- *Digital Certificates and Signatures*



Review of Symmetric Cryptography



- Confidentiality
 - Stream ciphers
 - Block ciphers
- Integrity
 - Cryptographic hash functions
- Limitation: **sender and receiver must share the same key**
 - Needs secure channel for key distribution
 - Impossible for two parties having no prior relationship
 - Needs many keys for n parties to communicate



Key Exchange



Traditionally, secure encrypted communication between two parties required that they first exchange **keys** by some secure physical channel, such as a paper key lists transported by a trusted courier.



Is this physical channel always available?



Security objective



- Breaking a public-key cryptographic primitive = solving a mathematical problem.
- A common metric: the size of the public key.
- When we increase the key size:
 - *the problem to solve becomes harder → more security*
 - *the computations also become heavier → less efficiency*
- The difficulty of the mathematical problem the attacker must solve depends on the algorithm used.
 - *Do we know the best algorithm?*



Design of cryptography algorithms



- For an algorithm that solves the mathematical problem, we have:
 - *time required to break it = $f(\text{key size})$.*
 - *The function f is the algorithm's complexity.*
- If we know the best algorithm:
 - *We can choose the right key size to guarantee a satisfactory difficulty level.*
 - *We can size standards and products accordingly.*
- If we are wrong and or better algorithm exists:
 - *The attack complexity is lower.*
 - *The key size we thought was sufficient is no longer enough.*
 - *Standards and recommendations must be updated.*





Improving complexity



There are two ways to improve complexity:

- *Improve the implementation of the best-known algorithm.*
 - *This is crucial when choosing key sizes;*
 - *Complex algorithms, there can be a large gap between an optimized implementation and a proof-of-concept one.*
- *Improve the algorithm's complexity itself.*
 - *This obviously brings the implementation question back into play.*

Best known algorithm for factoring an integer N : the **Number Field Sieve (NFS)**.

- What is an attacker's maximum computing capability?
 - *Today: about 2^{80} to 2^{100} operations.*
 - *In k years: on the order of $2^{(100+0.66k)}$.*

Typically (when feasible), “secure enough” is taken as 2^{128} .



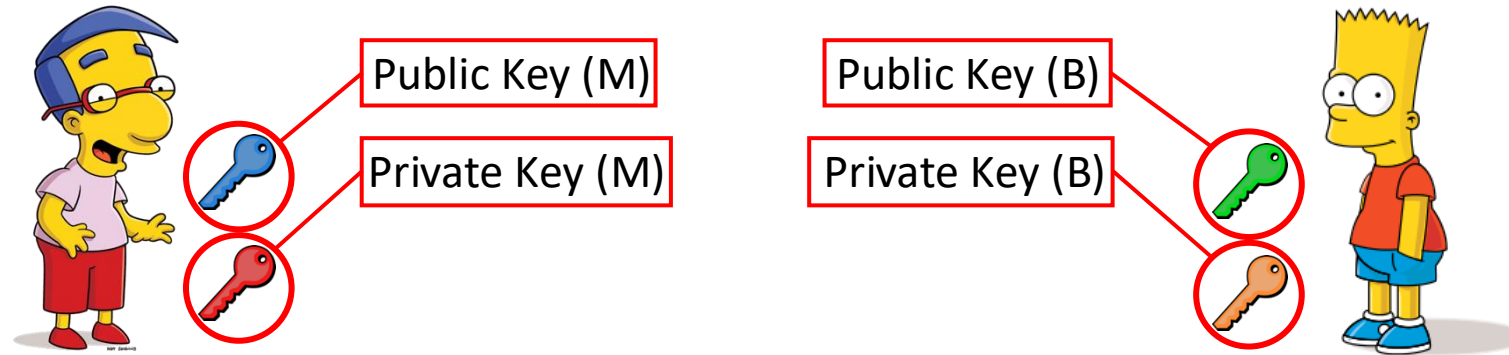
Complexity : Solving two problems Discrete Logarithm and Factoring



- **Rivest–Shamir–Adleman (1977):** RSA is based on the factoring principle.
Main idea: it is easy to find two large prime numbers p and q , but very hard to recover p and q from $n = p \cdot q$.
RSA remains the most popular public-key cryptosystem.
- **ElGamal (1984):** based on the Discrete Logarithm Problem (DLP).
Let G be an abelian group where every element has an inverse.
Main idea: it is easy to compute $h = g^x$ for a given x , but very hard to find x given h and g .
- **DSA signature algorithm:** discrete logarithm in finite fields.
- **ECDSA signature:** discrete logarithm on elliptic curves.



Asymmetric Cryptography



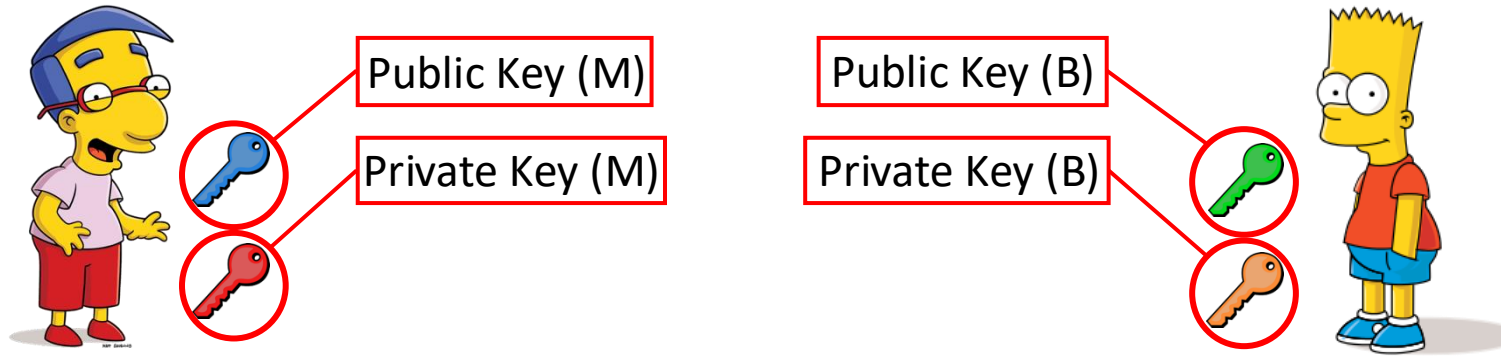
In asymmetric cryptography, both the sender and receiver have a public and a private key.

The public keys  ,  are publicly available.

The private keys  ,  have to be kept secret.



Asymmetric Cryptography



The Milhouse's public key  can reveal messages encrypted with the Milhouse's private key .

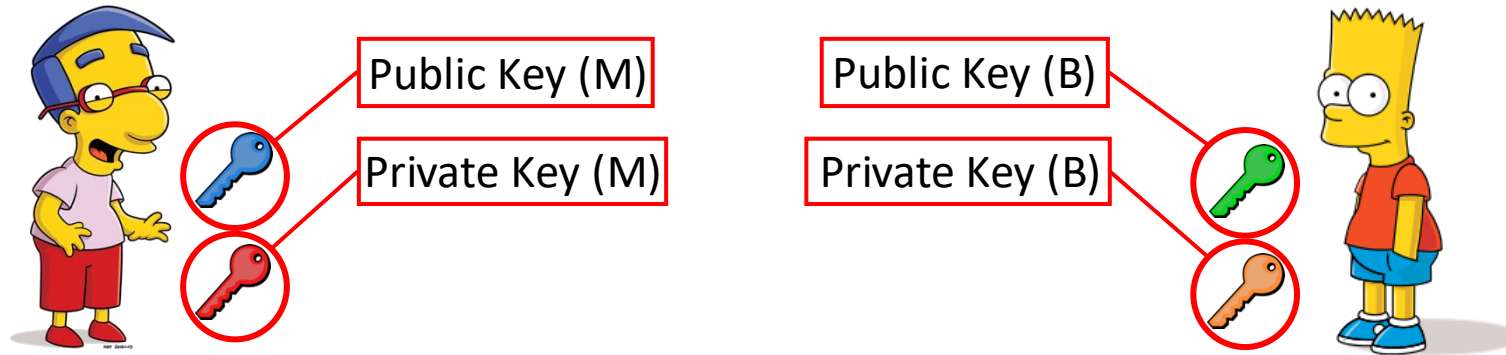
The Milhouse's private key  can reveal messages encrypted with the Milhouse's public key .

The Bart's public key  can reveal messages encrypted with the Bart's private key .

The Bart's private key  can reveal messages encrypted with the Bart's public key .



Asymmetric Cryptography - Confidentiality



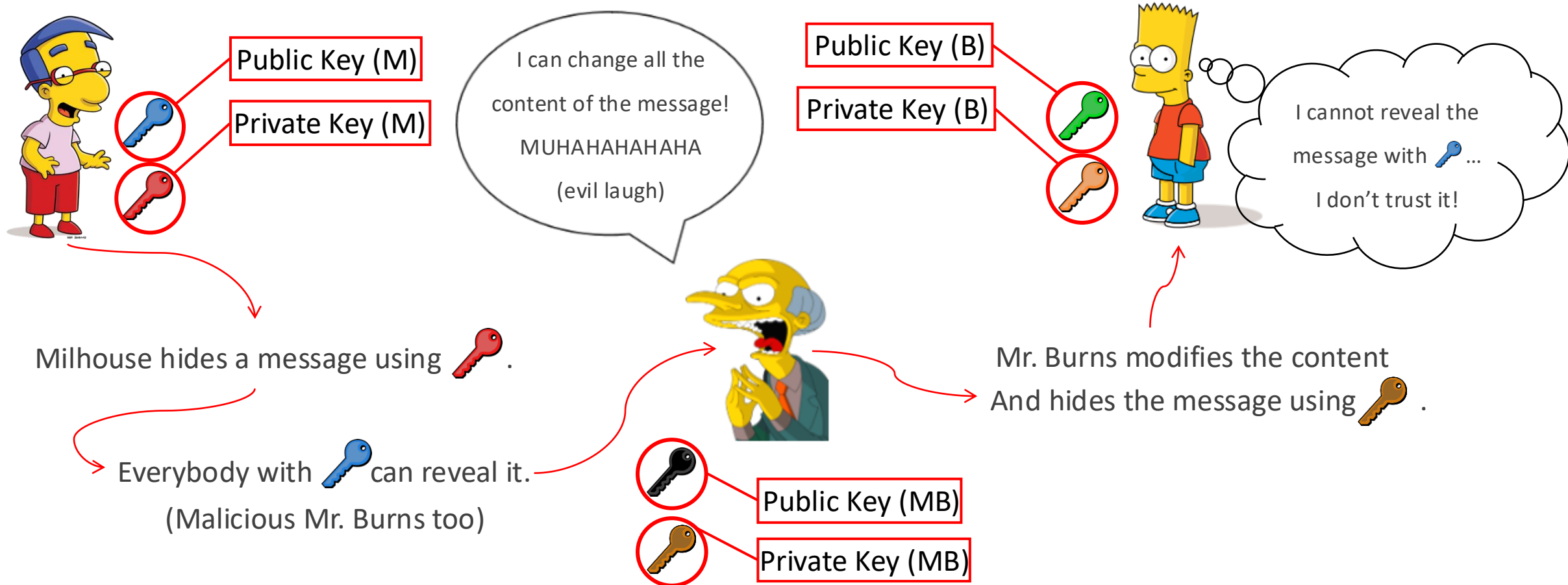
Milhouse hides a message using 🗝️.

Only Bart with 🗝️ can reveal it.

So, only Bart can read the message

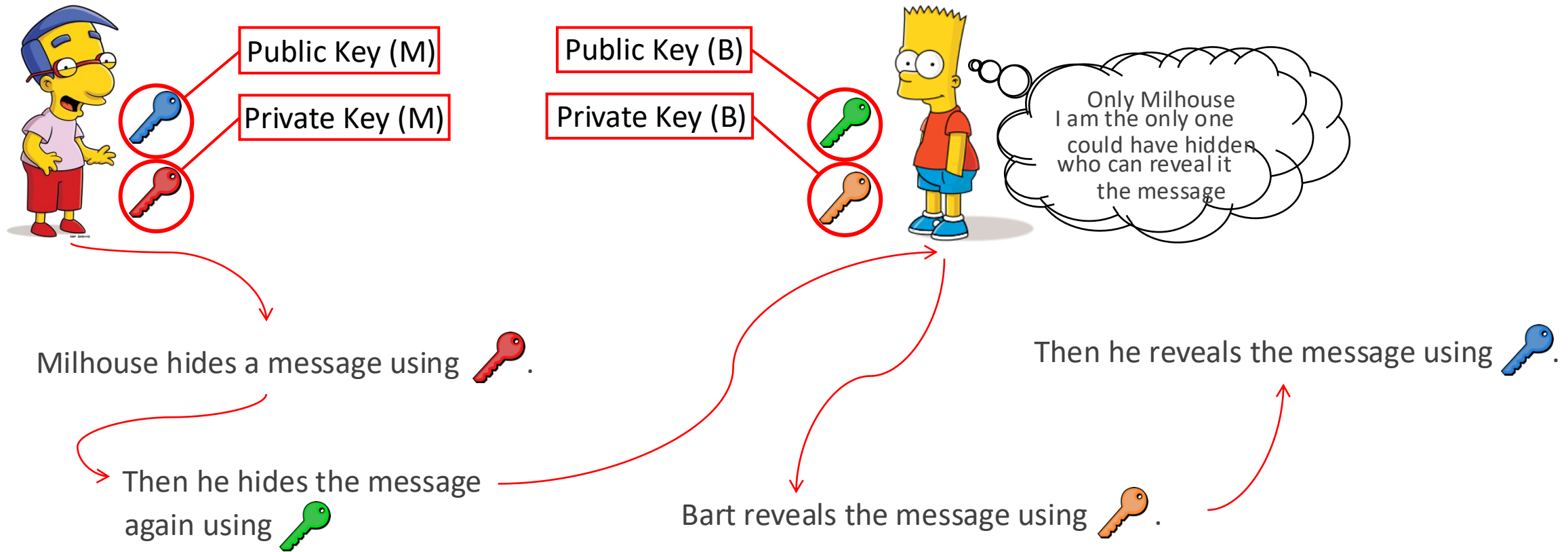


Asymmetric Cryptography - Integrity





Asymmetric Cryptography – Integrity and Confidentiality





Asymmetric Cryptography - Recap



- Each party has a PAIR (PubK, PrivK) of keys:
 - PubK is the public key, and used for encryption
 - PrivK is the private key, and used for decryption
- Knowing the public-key PubK, it is computationally **infeasible** to compute the private-key PrivK
- The public-key PubK may be made publicly available, e.g., in a publicly available directory
 - Many can encrypt, only one can decrypt



Asymmetric Cryptography - Advantages



- **Key distribution:**
 - Only the private key must be kept secret, the public key can be shared freely.
 - A key pairs is generated once and used forever (unless the private key is compromised!)
- **Many use cases:**
 - Confidentiality
 - Integrity
 - Authentication and non-repudiation (via digital signatures)



Asymmetric Cryptography - Disadvantages



Strength is dependent on the infeasibility of solving certain hard problems (discrete logarithm, factoring large numbers, etc.)

- Calculating $19 \times 31 = 589$ is easy, but given a large number and asked to determine its two prime factors is computationally difficult
- Note that in real systems, the product term is very large
- A mathematical breakthrough could provide a feasible way to solve the problem and break the encryption.
- Faster computers, quantum computers, etc. could facilitate breaking encryption.



Public Key Cryptography - Early History



- Public-key encryption was proposed in 1970 by James Ellis James Ellis
 - in a classified paper made public in 1997 by the British Governmental Communications Headquarters
- The concept is proposed in Diffie and Hellman (1976) “New Directions in Cryptography ”
 - public-key encryption schemes
 - Public key distribution systems
 - Diffie-Hellman key agreement protocol
 - digital signature digital signature
- Concept of digital signature is still originally due to Diffie & Hellman



Public Key Encryption Algorithms



- Almost all public-key encryption algorithms use either number theory and modular arithmetic, or elliptic curves
- **RSA**
 - based on the hardness of factoring large numbers
- **El Gamal**
 - Based on the hardness of solving discrete logarithm



RSA Algorithm



- Invented in **1978** by **Ron Rivest**, **Adi Shamir**, and **Leonard Adleman** – Published as R L Rivest, A Shamir, L Adleman, *"On Digital Signatures and Public Key Cryptosystems"*, *Communications of the ACM*, vol 21 no 2, pp 120-126, Feb 1978
- Security relies on the difficulty of factoring large composite numbers
- Essentially the same algorithm was discovered in 1973 by Clifford Cocks who works for the British intelligence



RSA Public Key Crypto System



Key Generation:

1. Select 2 large prime numbers of about the same size, **p** and **q**
2. Compute **n = pq**, and **$\phi(n) = (q-1)(p-1)$**
3. Select **e**: $1 < e < \phi(n)$ s.t. $\gcd(e, \phi(n)) = 1$
4. Compute **d**: $1 < d < \phi(n)$ s.t. $ed \bmod \phi(n) = 1$

Public key: (e, n)

Private key: d



RSA Public Key Crypto System



Encryption:

Given a message **M**, $0 < \mathbf{M} < \mathbf{n}$

use public key **(e, n)** to compute **C = M^e mod n**

Decryption:

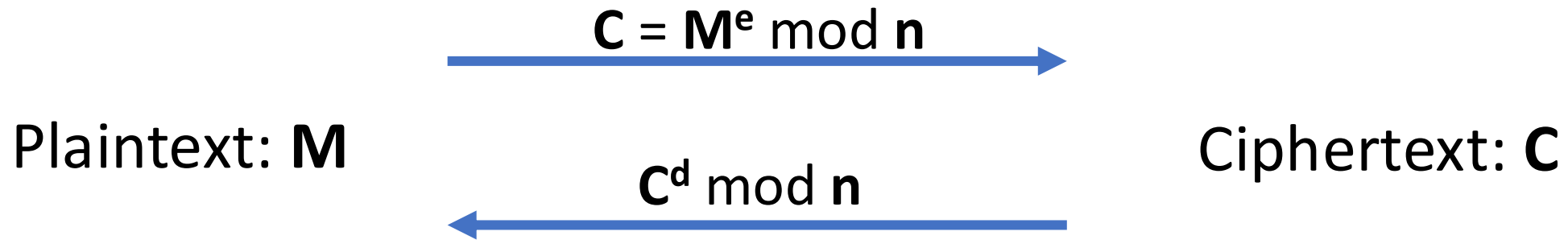
Given a ciphertext **C**

use private key **(d)** to compute

$$\mathbf{C}^d \bmod \mathbf{n} \rightarrow (\mathbf{M}^e \bmod \mathbf{n}) \bmod \mathbf{n} \rightarrow \mathbf{M}^{ed} \bmod \mathbf{n} = \mathbf{M}$$



RSA Properties



1. From **n**, difficult to compute **p**, **q**
2. From **(n, e)**, difficult to compute **d**
3. From **(n, e)** and **C**, difficult to compute **M**



RSA Example



1. $p = 11$ and $q = 7$
2. $n = pq, 11 * 7 = 77$
3. $\phi(n) = (q-1)(p-1) = (7-1)*(11-1) = 6*10 = 60$
4. Select e :
 1. between 1 and $\phi(n)$
 2. with no common divisors between e and $\phi(n)$
 3. $e = 37$
5. Select d :
 1. between 1 and $\phi(n)$ s.t. $37 * d \bmod 60 = 1$
 2. $d = 13$ ($37 * 13 = 481 \bmod 60 = 1$)
6. Let $M = 15$
7. Encryption:
 - $C = M^e \bmod n$
 - $C = 15^{37} \bmod 77 = 71$
8. Decryption:
 - $M = C^d \bmod n$
 - $M = 71^{13} \bmod 77 = 15$



RSA Example



1. $p = 11$ and $q = 7$
2. $n = pq, 11 * 7 = 77$
3. $\phi(n) = (q-1)(p-1) = (7-1)*(11-1) = 6*10 = 60$
4. Select e :
 1. between 1 and $\phi(n)$
 2. with no common divisors between e and $\phi(n)$
 3. $e = 37$
5. Select d :
 1. between 1 and $\phi(n)$ s.t. $37*d \bmod 60 = 1$
 2. $d = 13$ ($37*13 = 481 \bmod 60 = 1$)
6. Let $M = 15$
7. Encryption:
 - $C = M^e \bmod n$
 - $C = 15^{37} \bmod 77 = 71$
8. Decryption:
 - $M = C^d \bmod n$
 - $M = 71^{13} \bmod 77 = 15$



RSA Exercise



1. $p = 47$ and $q = 71$
2. Compute n and $\phi(n)$
3. Now pick a value of e which does not share a factor with $\phi(n)$
 - You can factorize a number here: <https://www.calculator.net/factor-calculator.html>
4. Now select a value of d , so that $(ed) \pmod{\phi(n)} = 1$
 - You can compute d here: <https://asecuritysite.com/encryption/inversemod>
5. Now for a message of $M=688$, calculate the cipher
6. Now decrypt your ciphertext

Did you get the value of your message back ($M=688$)? If not, you have made a mistake, so go back and check.



RSA – Security



- Security depends on the difficulty of factoring n
 - Factor $n \Rightarrow \phi(n) \Rightarrow$ compute d from $(e, \phi(n))$
- The length of $n=pq$ reflects the strength
 - 700-bit n factored in 2007
 - 768-bit n factored in 2009
- 1024 bit for minimal level of security today (likely to be breakable in near future)
- Minimal 2048 bits recommended for current usage
- NIST suggests 15360-bit RSA keys



Real World Usage of Public Key Encryption



- **Efficiency**

- asymmetric Cryptography encryption implies heavy performance issues
- far less efficient than symmetric cryptography (Not suitable for real-time or bulk encryption.)
- RSA speed is quadratic in key length

- **Solution:**

- Use Asymmetric Cryptography encryption to exchange the symmetric key over an unsecure channel.
- Then, use the shared key for the real communication using symmetric cryptography.



Real World Usage of Asymmetric Encryption - Integrity



Consider the real-life example where a person pays by credit card and signs a bill; the seller verifies that the signature on the bill is the same with the signature on the card

- Contracts: they are valid if they are signed
- Signatures provide non-repudiation.
 - ensuring that a party in a dispute cannot repudiate, or refute the validity of a statement or contract.
- Can we have a similar service in the electronic world?



Digital Signatures



- Digital Signature: a data string which associates a message with some originating entity.
- One party generates signature, many parties can verify.
- Digital Signature Scheme:
 - a signing algorithm: takes a message and a (private) signing key, outputs a signature
 - a verification algorithm: takes a (public) verification key, a message, and a signature
- Provides:
 - Authentication, Data integrity, Non-Repudiation

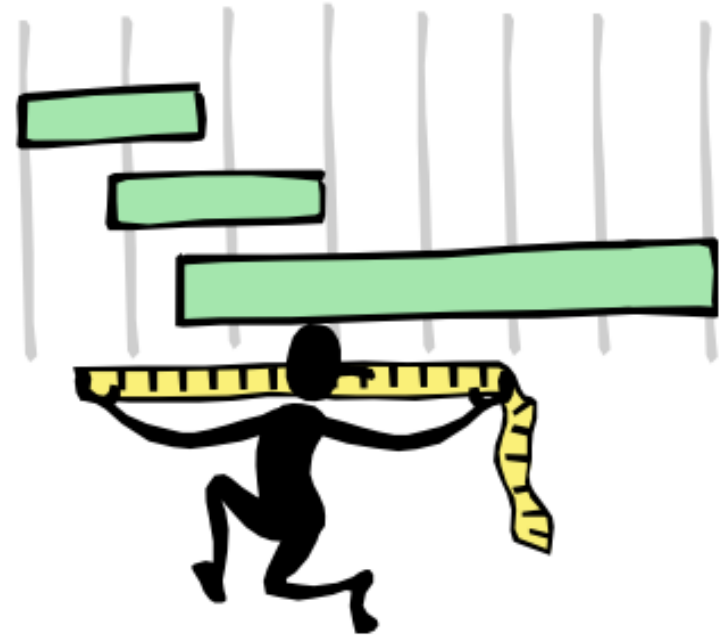


Digital Signatures



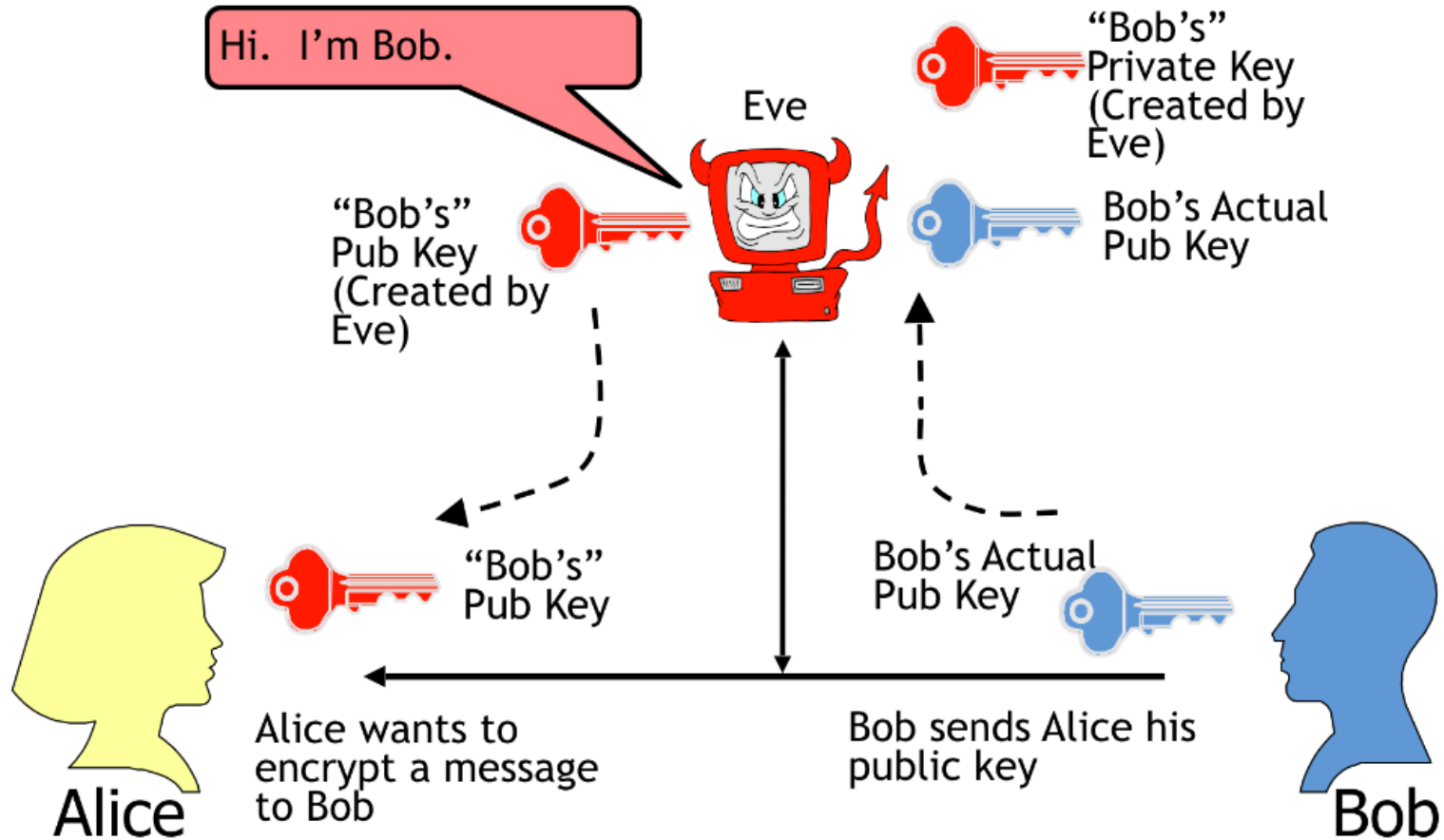
Very often digital signatures are used with hash functions

- The hash of a message is signed, instead of the message.



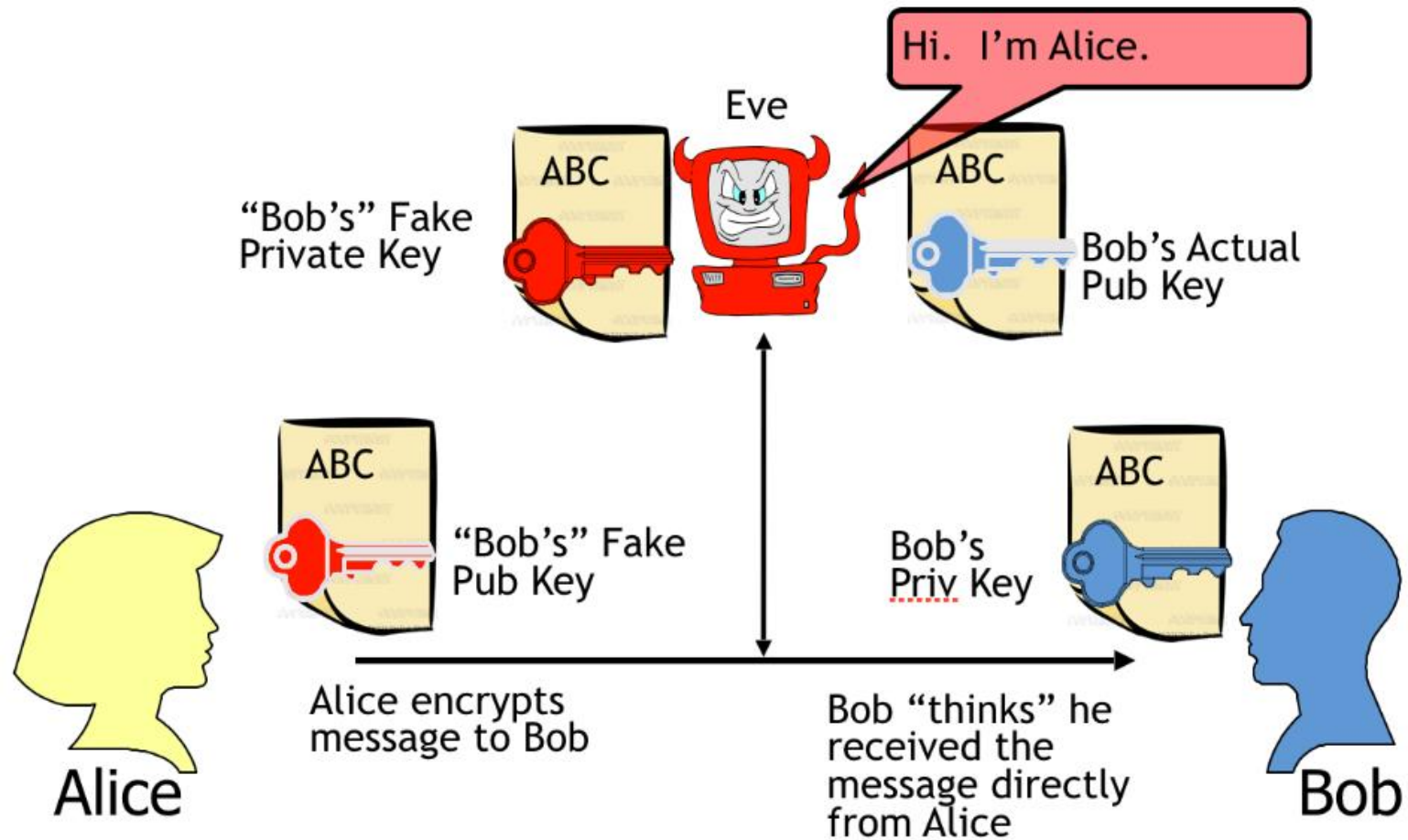


MitM Attack Against Asymmetric Key Systems





MitM Attack Against Asymmetric Key Systems





Asymmetric Key Systems - Problem



- How do we know we are really talking to the right party? (Anyone can send you a public key to use)
- A digital signature only ties a message to a private key, not to a person.
- We need a way to bind a public/private key pair to a specific individual.



Asymmetric Key Systems - Solution



Public Key Infrastructure (PKI)

- A trusted third party, called **certificate authority** (CA), vouches for the individuals and their keys.
- A **registration authority** (RA) issues digital certificates that bind key pairs to identities.
- Certificates can be created for people, organizations, or specific computers.
- The PKI includes mechanisms to issue, revoke, and verify certificates.



Public Key Infrastructure (PKI)



- The issuer digitally signs the certificates, allowing us to verify their authenticity and integrity.
- There may be multiple layers of certificates verifying others in a 'tree of trust' or 'web of trust'.
- A CA has one or more root certificates that we must trust implicitly.
- Certificates have an expiration date. The CA may issue a certificate revocation list (CRL) of certificates that have not expired but should not be trusted.



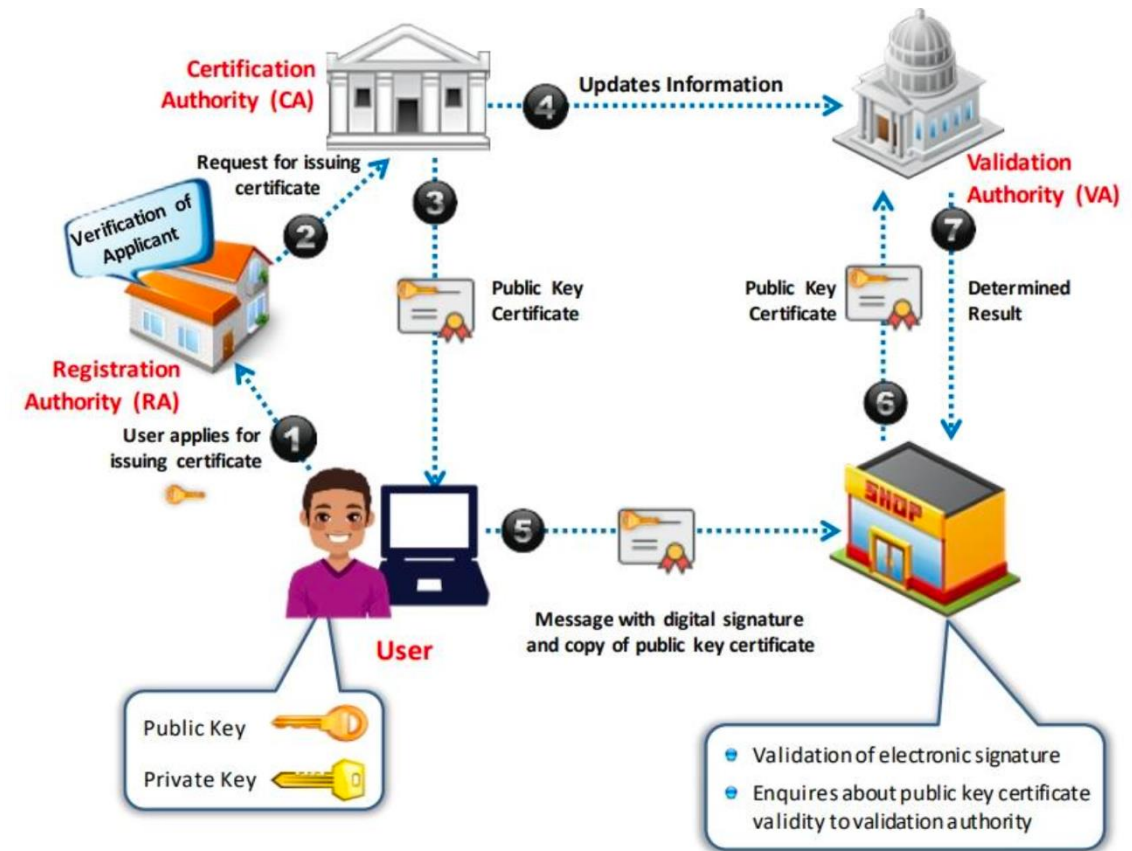
Public Key Infrastructure



- PKI is asset of hardware, software, people, policies, and procedures required to create, manage, distribute, use, store, and revoke digital certificates

Components of PKI

- **Certificate Management System:** Generates, distributes, stores, and verifies certificates
- **Digital Certificates:** Establish people's credentials in online transactions
- **Validation Authority (VA):** Stores certificates (with their public keys)
- **Certificate Authority (CA):** Issues and verifies digital certificates
- **End User:** Requests, manages, and uses certificates
- **Registration Authority (RA):** Acts as the verifier for the certificate authority





Public Key Certificates



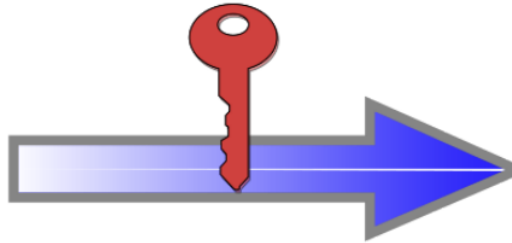
Identity Information and
Public Key of Mario Rossi

Name: *Mario Rossi*
Organization: *Wikimedia*
Address: *via*
Country: *United States*



Public Key
of
Mario Rossi

Certificate Authority
verifies the identity of Mario Rossi
and encrypts with its Private Key



Certificate of Mario Rossi

Name: *Mario Rossi*
Organization: *Wikimedia*
Address: *via*
Country: *United States*
Validity: *1997/07/01 - 2047/06/30*



Public Key
of
Mario Rossi

Digital Signature
of the Certificate Authority

Digitally Signed by
Certificate Authority



X.509 Certificates



- Defines framework for authentication services
- The most widely accepted format for certificates is defined by the ITU-T X.509 version 3 international standards.
- The certificates are encoded using OSI ASN.1 DER.
- Used by numerous applications: SSL, IPSec, SET

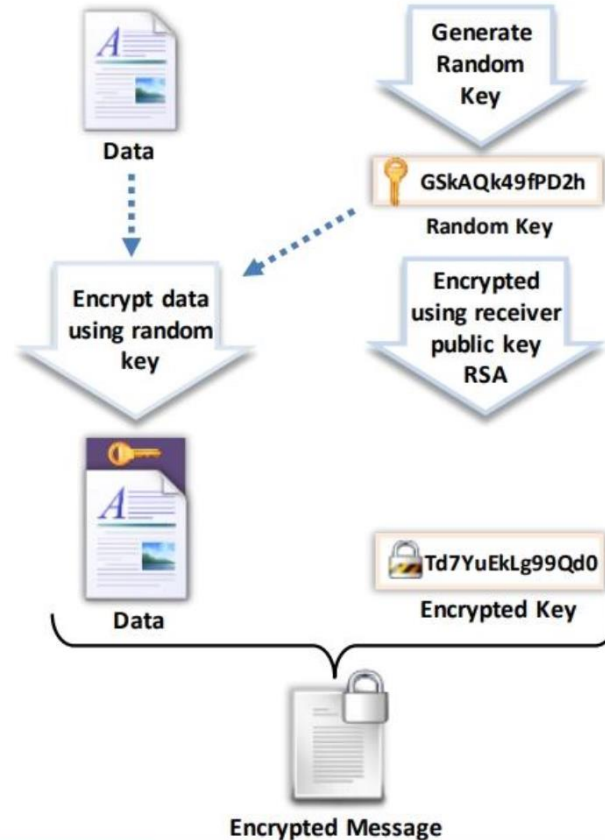


Pretty Good Privacy (PGP)

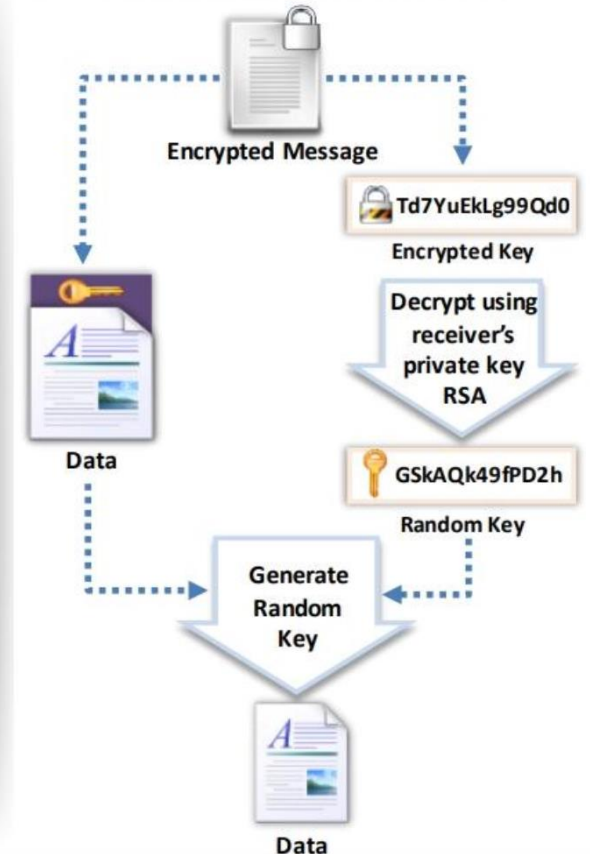


- PGP is a protocol used to **encrypt** and **decrypt** data that provides **authentication** and **cryptographic privacy**
- It is often used for data **compression, digital signing, encryption and decryption of messages, emails, files, directories**, and to enhance the privacy of email communications
- It combines the best features of both **conventional** and **public key cryptography** and is therefore known as a **hybrid cryptosystem**

PGP Encryption



PGP Decryption

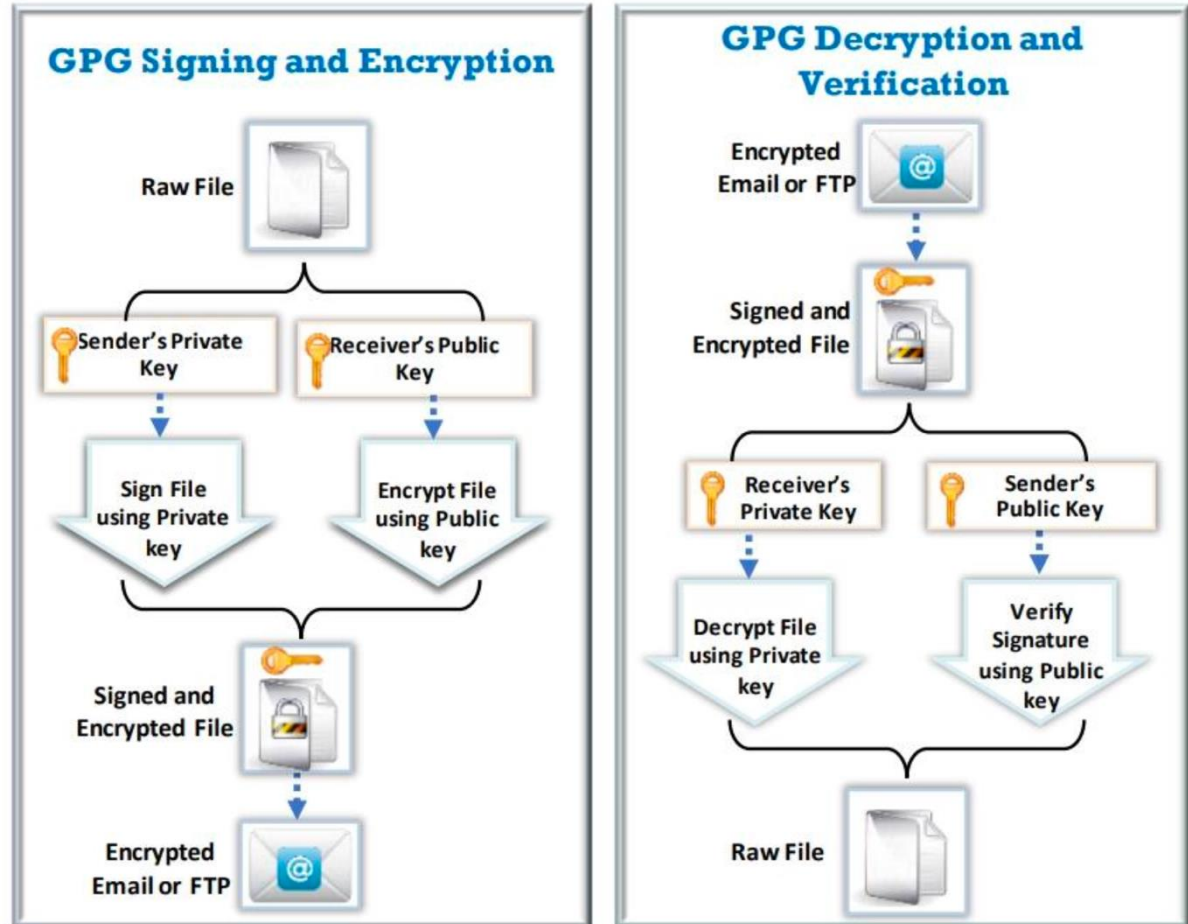




GNU Privacy Guard (GPG)



- GPG is a **software replacement of PGP** and free implementation of the OpenPGP standard
- GPG is also called **hybrid encryption software** as it uses both symmetric key cryptography and asymmetric key cryptography
- It also supports S/MIME and Secure Shell (SSH)

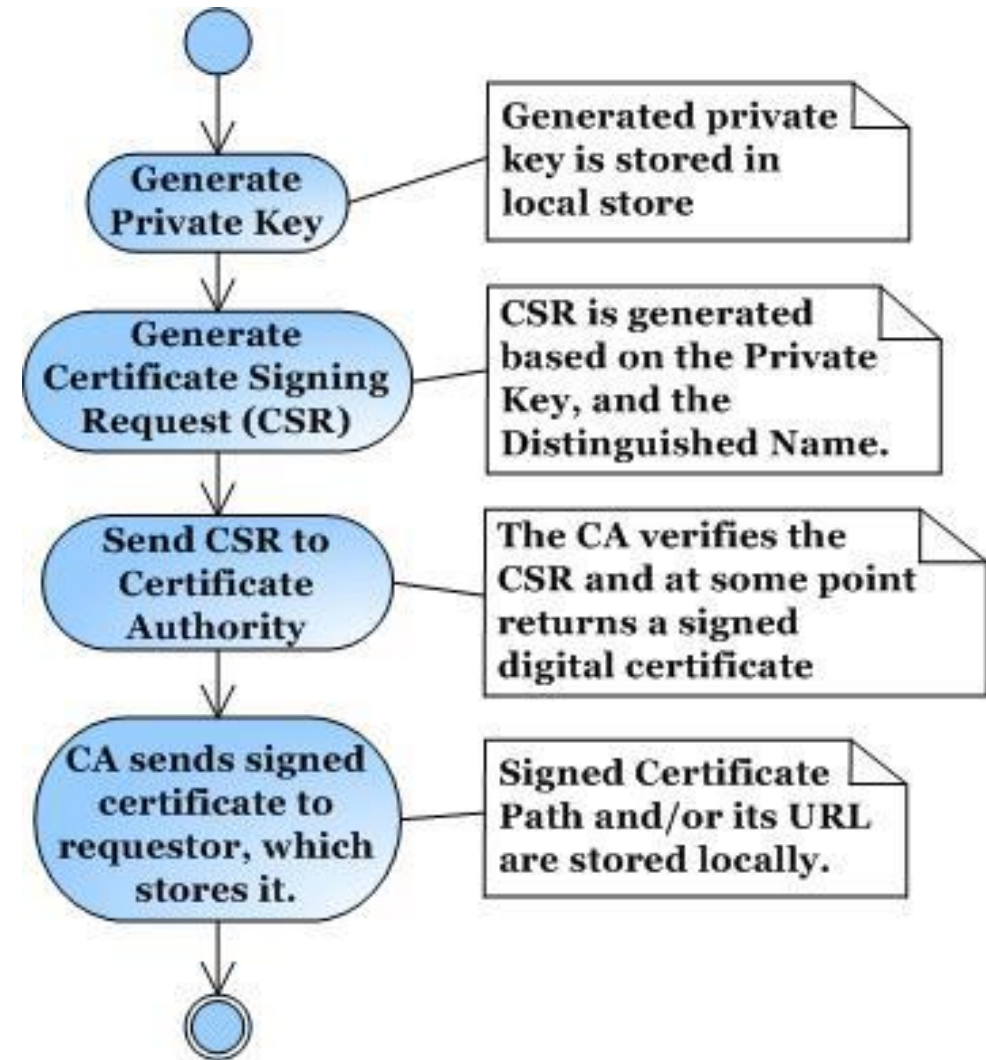




How to Obtain a Certificate?



- Define your own CA (use openssl or Java Keytool)
Certificates unlikely to be accepted by others
- Obtain certificates from one of the vendors: VeriSign, Thawte, and many others





CAs and Trust



- Certificates are trusted if signature of CA verifies
- Chain of CA's can be formed, head CA is called root CA
- In order to verify the signature, the public key of the root CA should be obtained.
- TRUST is centralized (to root CA's) and hierarchical
- What bad things can happen if the root CA system is compromised?



CAs – Chain of Trust



End-entity Certificate

Owner's name
Owner's public key
Issuer's (CA's) name
Issuer's signature

reference

Intermediate Certificate

Owner's (CA's) name
Owner's public key
Issuer's (root CA's) name
Issuer's signature

reference

Root CA's name
Root CA's public key
Root CA's signature

Root Certificate

sign

sign

self-sign

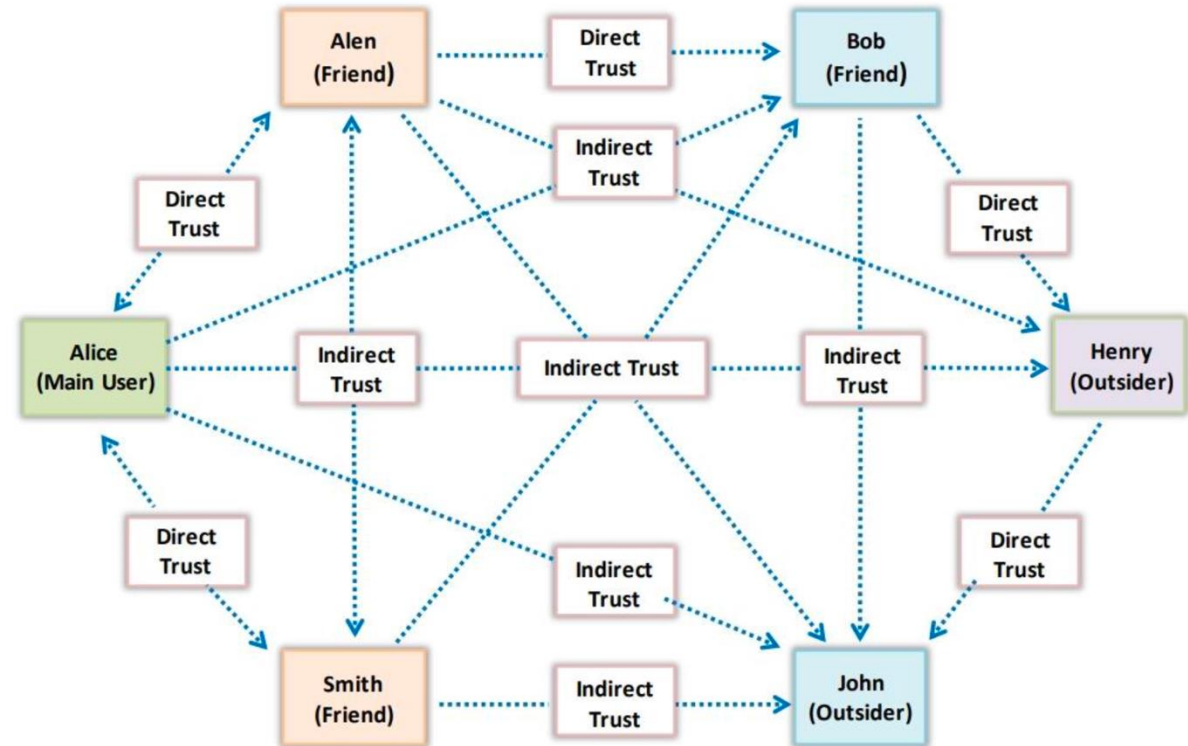




Web of Trust (WOT)



- Web of trust (WoT) is **a trust model of PGP, OpenPGP, and GnuPG systems**
- Everyone in the network is a Certificate Authority (CA) and signs for other trusted entities
- WoT is a **chain of a network** in which individuals intermediately validate each other's certificates using their signatures
- Every user in the network has a **ring of public keys** to encrypt the data, and they introduce many other users whom they trust



Do you have any questions?



rc3.kaust.edu.sa



Follow us @rc3kaust

